

1. Access Modifiers

Access modifiers define the **visibility** of classes, methods, and variables. They act as "security gates" for your code.

Modifier	Scope	Use Case
public	Everywhere	APIs and shared tools
private	Within Class	Sensitive data (Encapsulation)
protected	Package + Subclasses	Inheritance structures
<i>Default</i>	Same Package	Internal package utilities

2. Visibility Example

```
class Person {
    public String name = "John"; // Open access
    private int age = 30;        // Hidden from outside
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        System.out.println(p.name); // Works!
        // System.out.println(p.age); // COMPILE ERROR: age is private
    }
}
```

3. Non-Access Modifiers

These keywords do not change visibility but provide specialized **behavioral attributes** to the code.

Modifier	Functional Description
static	Belongs to the class (exists without an object).
final	Fixes the value; prevents overriding or modification.
abstract	Designates a class/method that must be implemented elsewhere.
synchronized	Forces threads to access code one at a time.

Security Best Practices

- Always start with **private** and only increase access if absolutely necessary.
- Use **final** for constants (like PI) to prevent accidental bugs.
- Use **static** for utility methods that don't need object data.